



# Security Audit Report

DZap



Version: Final

Date: 15th June 2025

# Table of Contents

|                                                |           |
|------------------------------------------------|-----------|
| <b>Table of Contents</b>                       | <b>2</b>  |
| <b>License</b>                                 | <b>3</b>  |
| <b>Disclaimer</b>                              | <b>4</b>  |
| <b>Introduction</b>                            | <b>5</b>  |
| <b>Codebases Submitted for the Audit</b>       | <b>6</b>  |
| <b>How to Read This Report</b>                 | <b>7</b>  |
| <b>Overview</b>                                | <b>8</b>  |
| <b>Summary of Findings</b>                     | <b>9</b>  |
| <b>Detailed Findings</b>                       | <b>10</b> |
| Critical Severity Issues                       | 10        |
| Signature Malleability in LibValidator         | 10        |
| High Severity Issues                           | 11        |
| Permit2 Integration Vulnerabilities            | 11        |
| Insufficient Balance Validation in LibAsset    | 12        |
| Quorum Manipulation in DZapWalletManager       | 13        |
| Signature Replay Across Different Contexts     | 14        |
| Medium Severity Issues                         | 15        |
| Inadequate Error Handling in Permit Operations | 15        |
| Factory Address Collision Vulnerability        | 16        |
| Inadequate Nonce Management                    | 17        |
| Low Severity Issues                            | 18        |
| Insufficient Event Logging                     | 18        |
| Missing Zero Amount Checks                     | 19        |

# License

THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](#).

# Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

# Introduction

## Purpose of this report

0xCommit has been engaged by **DZap -Contracts** to perform a security audit of several Solana Programs components.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine solana program bugs, which might lead to unexpected behaviour.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

# Codebases Submitted for the Audit

The audit has been performed on the following GitHub repositories:

| Version | Source                                                                                                                                                                                                      |
|---------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Final   | <a href="https://github.com/DZapIO/DZapZappingContracts/commit/3d4c6f189758d6bcacda0e2adb059bc357314740">https://github.com/DZapIO/DZapZappingContracts/commit/3d4c6f189758d6bcacda0e2adb059bc357314740</a> |

# How to Read This Report

This report classifies the issues found into the following severity categories:

| Severity             | Description                                                                                                                                                                                                                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Critical</b>      | A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.                                                                                                                                                                                          |
| <b>Major</b>         | A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.                                                                                                                                                                                                    |
| <b>Minor</b>         | A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.                                                                                                                                        |
| <b>Informational</b> | Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share. |

The status of an issue can be one of the following: **Pending**, **Acknowledged**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

# Overview

## Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line by line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
  - a. Race condition analysis
  - b. Under-/overflow issues
  - c. Key management vulnerabilities
4. Report preparation

# Summary of Findings

| Sr. No. | Description                                    | Severity   | Status         |
|---------|------------------------------------------------|------------|----------------|
| 1       | Signature Malleability in LibValidator         | Critical ▾ | Resolved ▾     |
| 2       | Permit2 Integration Vulnerabilities            | High ▾     | Resolved ▾     |
| 3       | Insufficient Balance Validation in LibAsset    | High ▾     | Acknowledged ▾ |
| 4       | Quorum Manipulation in DZapWalletManager       | High ▾     | Resolved ▾     |
| 5       | Signature Replay Across Different Contexts     | High ▾     | Resolved ▾     |
| 6       | Inadequate Error Handling in Permit Operations | Medium ▾   | Resolved ▾     |
| 7       | Factory Address Collision Vulnerability        | Medium ▾   | Resolved ▾     |
| 8       | Inadequate Nonce Management                    | Critical ▾ | Resolved ▾     |
| 9       | Insufficient Event Logging                     | Low ▾      | Resolved ▾     |
| 10      | Missing Zero Amount Checks                     | Low ▾      | Resolved ▾     |

# Detailed Findings

## Critical Severity Issues

### Signature Malleability in LibValidator

Severity: CRITICAL Files: **LibValidator.sol**

**Issue Description:** The signature validation doesn't check for signature malleability:

```
assembly {  
    r := mload(add(_signatures, add(sigOffset, 32)))  
    s := mload(add(_signatures, add(sigOffset, 64)))  
    v := byte(0, mload(add(_signatures, add(sigOffset, 96))))  
}
```

```
address validator = ECDSA.recover(_hash, v, r, s);  
require(_validators[validator], NotAValidator(i, validator));
```

#### Risk:

- **Signature replay attacks** using malleable signatures
- **Bypass of duplicate signature detection**
- **Double spending** possibilities

#### Remediation:

1. **Check s value** is in lower half of curve order
2. **Use OpenZeppelin's ECDSA.recover** with malleability protection
3. **Implement signature canonicalization**

## Status

Resolved ▾

## High Severity Issues

### Permit2 Integration Vulnerabilities

Severity: HIGH Files: **LibPermit.sol**

**Issue Description:** The Permit2 integration doesn't validate allowance amounts properly:

```
function permit2ApproveAndTransfer(address _permit2, address _from, address _to,
uint160 _amount, address _token, bytes memory _data) internal {
    if (_data.length != 0) {
        (, uint48 nonce, uint48 expiration, uint256 sigDeadline, bytes memory signature) =
abi.decode(_data, (uint160, uint48, uint48, uint256, bytes));
        IPermit2(_permit2).permit(_from,
IPermit2.PermittSingle(IPermit2.PermittDetails(_token, _amount, expiration, nonce), _to,
sigDeadline), signature);
    }
    IPermit2(_permit2).transferFrom(_from, _to, _amount, _token);
}
```

#### Risk:

- **Unauthorized token transfers** beyond permitted amounts
- **Nonce manipulation** attacks
- **Expired permit usage**

#### Remediation:

1. **Validate permit amounts** match transfer amounts
2. **Check permit expiration** before use
3. **Implement nonce tracking** to prevent replay

## Status

Resolved ▾

## Insufficient Balance Validation in LibAsset

Severity: HIGH Files: **LibAsset.sol**

**Issue Description:** ERC20 transfer validation has potential bypass:

```
function transferFromERC20(address _token, address _from, address _to, uint256
_amount) internal {
    IERC20 token = IERC20(_token);
    uint256 prevBalance = token.balanceOf(_to);
    SafeERC20.safeTransferFrom(token, _from, _to, _amount);
    require(token.balanceOf(_to) - prevBalance == _amount, InvalidAmount());
}
```

**Risk:**

- **Fee-on-transfer token** compatibility issues
- **Deflationary token** handling problems
- **Transfer amount validation** bypass

**Remediation:**

1. **Handle fee-on-transfer tokens** explicitly
2. **Use actual received amounts** for validation
3. **Implement token blacklist** for incompatible tokens

## Status

Acknowledged ▾

## Quorum Manipulation in DZapWalletManager

Severity: HIGH Files: **DZapWalletManager.sol**

**Issue Description:** The quorum can be changed without timelock protection:

```
function setQuorum(uint8 _quorum) external onlyOwner {  
    require(_quorum >= MIN_QUORUM, QuorumTooLow());  
    quorum = _quorum;  
    emit QuorumUpdated(_quorum);  
}
```

**Risk:**

- **Security downgrade** by reducing quorum
- **Centralization** through quorum manipulation
- **Emergency bypass** of security measures

**Remediation:**

1. **Implement timelock** for quorum changes
2. **Require validator consensus** for quorum reduction
3. **Set maximum quorum** to prevent DOS

## Status

Resolved ▾

## Signature Replay Across Different Contexts

Severity: HIGH Files: **DZapWallet.sol**

**Issue Description:** The domain separator is computed during initialization but doesn't account for potential proxy patterns:

```
function initialize(address _user) public initializer {  
    _setOwner(_user);  
    __ReentrancyGuard_init();  
    _DOMAIN_SEPARATOR = keccak256(abi.encode(_DOMAIN_TYPEHASH,  
keccak256(bytes("DZapWallet")), block.chainid, address(this)));  
}
```

### Risk:

- **Cross-chain signature replay** if deployed on multiple chains
- **Cross-wallet signature replay** between different wallet instances
- **Potential bypass** of signature verification

### Remediation:

1. Include **wallet-specific salt** in domain separator
2. Include **user address** in domain separator
3. Consider using **CREATE2** deterministic addresses in domain separator

## Status

Resolved ▾

## Medium Severity Issues

### Inadequate Error Handling in Permit Operations

Severity: MEDIUM Files: **LibPermit.sol**

**Issue Description:** Permit operations can fail silently in some edge cases:

```
function permit(address _token, address _from, address _to, uint256 _amount, bytes
memory _data) internal {
    if (_data.length == 32 * 7) {
        (, , uint256 deadline, uint8 v, bytes32 r, bytes32 s) = abi.decode(_data, (address,
address, uint256, uint256, uint8, bytes32, bytes32));
        IERC20Permit(_token).permit(_from, _to, _amount + 1, deadline, v, r, s);
    } else revert InvalidPermitData();
}
```

**Risk:**

- **Silent permit failures**
- **Unexpected behavior** with non-standard permits
- **Transaction failures** without clear error messages

**Remediation:**

1. **Implement comprehensive error handling**
2. **Add permit validation** before execution
3. **Use try-catch** for permit operations

## Status

Resolved ▾

## Factory Address Collision Vulnerability

Severity: MEDIUM Files: **DZapWalletFactory.sol**

**Issue Description:** The factory uses simple keccak256 hashing for salt generation:

```
function _createSalt(address _user, string memory _label) private pure returns (bytes32) {  
    return keccak256(abi.encodePacked(_user, _label));  
}
```

### Risk:

- **Hash collision** potential with specific user/label combinations
- **Wallet deployment conflicts**
- **Denial of service** for legitimate deployments

### Remediation:

1. Use **abi.encode** instead of **abi.encodePacked**
2. Include additional entropy (e.g., block.timestamp, nonce)
3. Add **collision detection** and retry mechanism

## Status

Resolved ▾

## Inadequate Nonce Management

Severity: MEDIUM Files: **DZapWallet.sol**

**Issue Description:** Nonce checking happens after verification but before execution:

```
function execute(bytes32 _txId, uint256 _deadline, uint256 _nonce, bytes calldata _data,
bytes calldata _validatorSignatures) external payable onlyAuthorizedExecutorOrOwner
whenNotPaused nonReentrant {
    _verify(_txId, _deadline, _nonce, _data, _validatorSignatures);
    nonces[_nonce] = true; // Set after verification
```

### Risk:

- **Race conditions** in high-frequency usage
- **Nonce conflicts** between different executors
- **Potential replay** in edge cases

### Remediation:

1. Use **atomic nonce increment** pattern
2. Implement **per-executor nonces**
3. Add **nonce expiration** mechanism

## Status

Resolved ▾

## Low Severity Issues

### Insufficient Event Logging

**Severity:** LOW **Files:** All contracts

**Issue Description:** Critical state changes lack comprehensive event emission.

**Remediation:**

- Add events for all admin actions
- Include relevant parameters in events
- Implement event indexing for better querying

### Status

Resolved ▾

## Missing Zero Amount Checks

Severity: LOW Files: **DZapZapCore.sol**

**Issue Description:** Token transfer functions don't validate non-zero amounts.

### Remediation:

- Add zero amount validation
- Implement minimum transfer thresholds
- Skip processing for zero amounts

## Status

Resolved ▾